Laundry Management System Using a Queue and a Binary Search Tree

Submitted By

Student Name	Student ID
MD Sajid Ali	253-15-147
MD Junaid Al Habib Siam	253-15-769
Manosh Sarkar	253-15-264

MINI LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course **CSEXXX: Data Structures in the Computer Science and Engineering Department**



DAFFODIL INTERNATIONAL UNIVERSITY
Dhaka, Bangladesh

August 6, 2026

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of [Course Teacher's Name], [Designation], Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

Zannatul Mawa Koli
Lecturer
Department of Computer Science and Engineering
Daffodil International University

Submitted by

MD Sajid Ali
Student ID: 253-15-147
Dept. of CSE, DIU

MD Junaid Al Habib Siam
Student ID: 253-15-769
Dept. of CSE, DIU

Manosh Sarkar
Student ID: 253-15-264
Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course has course outcomes as following:

Table 1: Course Outcome Statements

CO's	Statements
CO1	Define and relate the linear and non-linear data structures — arrays, linked lists, stacks, queues and trees — together with the operations defined on them and the relationships among them needed for solving a specific problem.
CO2	Formulate knowledge of data structures and the C programming language in problem solving, using pointers, dynamic memory allocation and recursion.
CO3	Analyze the time and space complexity of alternative data-structure choices in order to present and justify a solution to a specific problem.
CO4	Develop solutions for real-world complex problems by selecting, implementing and integrating appropriate data structures, while evaluating their effectiveness against measured performance.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Blooms	KP	CEP
CO1	PO1	C1, C2	KP3	EP1, EP3
CO2	PO2	C2	KP3	EP1, EP3
CO3	PO3	C4, A1	KP3	EP1, EP2
CO4	PO3	C3, C6, A3, P3	KP4	EP1, EP3

The mapping justification of this table is provided in section 4.3.1, 4.3.2 and 4.3.3.

Table of Contents

Declaration i

Course & Program Outcome ii

1 Introduction 1

1.1 Introduction 1

1.2 Motivation 1

1.3 Objectives 1

1.4 Feasibility Study 2

1.5 Gap Analysis 2

1.6 Project Outcome 2

2 Proposed Methodology/Architecture 3

2.1 Requirement Analysis & Design Specification 3

2.1.1 Overview 3

2.1.2 Proposed Methodology / System Design 3

2.1.3 UI Design 5

2.2 Overall Project Plan 6

3 Implementation and Results 7

3.1 Implementation 7

3.2 Performance Analysis 8

3.3 Results and Discussion 9

4 Engineering Standards and Mapping 12

4.1 Impact on Society, Environment and Sustainability 12

4.1.1 Impact on Life 12

4.1.2 Impact on Society & Environment 12

4.1.3 Ethical Aspects 12

4.1.4 Sustainability Plan 12

4.2 Project Management and Team Work 12

4.3 Complex Engineering Problem 13

4.3.1 Mapping of Program Outcome 13

4.3.2 Complex Problem Solving 14

4.3.3 Engineering Activities 14

5 Conclusion 15

5.1 Summary 15

5.2 Limitation 15

5.3 Future Work 15

References 16

Chapter 1
Introduction

This chapter introduces the service problem that motivated the project, states the computational reasoning behind the chosen data structures, lists the objectives, reviews comparable systems and identifies the gap that the work addresses.

1.1 Introduction

Residential halls, hostels and small hotels in Bangladesh almost always run an in-house laundry counter, and the workflow is entirely manual. A guest hands over a bundle, the attendant writes the room number, the guest name and the piece count in a register, and a hand-written slip is torn off as a receipt. When the washing is finished the bundle goes on a shelf, and when the guest returns the attendant reads down the register until the right row is found.

That manual process fails in three predictable ways. First, the register does not enforce any processing order, so a bundle handed in early can sit untouched while a later one is washed, which is the most common source of complaints. Second, retrieval is a linear search: with sixty or seventy bundles on the shelf the attendant must scan a large part of the register for every enquiry. Third, the piece count and the charge exist only as ink on paper, so a disputed bill has no reliable record behind it.

The problem statement is therefore: design an offline, single-operator console application in C that guarantees first-come-first-served processing, retrieves any finished bundle by room number in sub-linear time, and produces a machine-generated ticket and bill so the charge is never in dispute.

1.2 Motivation

The computational motivation is that a single real workflow contains two genuinely different access patterns, and no single data structure serves both well.

- **Intake and washing are strictly sequential.** An order handed over first must be washed first. This is the textbook definition of a first-in-first-out queue, and a queue implemented as a linked list gives O(1) insertion at the rear and O(1) removal from the front.
- **Retrieval is key based and unordered.** A guest arrives quoting a room number, and the room numbers are requested in no predictable sequence. A queue is useless here because it can only be examined from the front; an array would need a linear scan or a costly re-sort after every insertion. A binary search tree keyed on the room number gives expected O(log n) search, insertion and deletion, and — at no extra cost — an in-order traversal that prints the shelf sorted by room number.

Recognising that the correct answer is to use both structures, joined by a single dequeue-then-insert step, is exactly the design judgement a data structures course is meant to develop. Building it in C rather than in a language with built-in collections also forced us to write the pointer manipulation, recursion and dynamic allocation ourselves.

1.3 Objectives

The objectives of this project are stated below.

- To model a hostel laundry counter as a two-stage pipeline in which pending orders are held in a FIFO queue and finished bundles are held in a binary search tree.
- To implement the queue as a singly linked list with front and rear pointers, supporting enqueue and dequeue in constant time.
- To implement the storage tree with recursive insertion, search, in-order traversal and deletion, including the three standard deletion cases.
- To generate a unique token number, a printed ticket at intake and an itemised bill at delivery, so that billing is automatic and traceable.
- To make the console interface readable and self-explanatory through framed panels, colour-coded status and input validation, so that an untrained attendant can operate it.
- To measure the lookup cost of the tree against a linear scan and to verify empirically that the observed behaviour matches the predicted asymptotic complexity.

- To ensure that every dynamically allocated node is released, so that the program can run for a long session without leaking memory.

1.4 Feasibility Study

Commercial laundry management products exist and are mature: point-of-sale packages used by dry-cleaning chains handle order intake, barcode tagging, route delivery and payment, and hotel property management systems expose laundry as a billable folio item. Academically, service-counter simulations built on queues are a standard illustration in algorithm texts [1], [5], and the binary search tree is the classical structure presented for ordered dictionary operations [2], [4], [8].

Those products are, however, subscription-priced and network-dependent. A hall that processes forty bundles a day cannot justify a recurring licence fee, and an intermittent campus connection makes a cloud-hosted product unreliable at exactly the counter where it is needed.

Technical feasibility

The system is written in standard C [7] and uses only the standard library. It compiles with GCC on Linux and with MinGW or MSVC on Windows; the only platform-specific code enables UTF-8 output and virtual-terminal escape sequences on the Windows console. It runs on any lab computer with no installation step.

Operational feasibility

The interface is a numbered menu with six operations. Every prompt states what is expected, invalid input is rejected without crashing, and the ticket and bill are produced automatically, so an attendant needs only a few minutes of instruction.

Economic feasibility

Development used free tools and existing hardware, so there is no licence, server or recurring cost. This is analysed further in section 4.2.

1.5 Gap Analysis

Table 1.1 compares the options available to a hall laundry counter today.

Table 1.1: Comparison of existing approaches with the proposed system

Criterion	Paper register	Spreadsheet	Commercial POS	Proposed system
Recurring cost	None	Licence	Subscription	None
Works fully offline	Yes	Yes	Usually not	Yes
FIFO order enforced	No	No	Yes	Yes, by construction
Lookup cost	O(n) by eye	O(n) or manual sort	Indexed	O(log n) expected
Automatic billing	No	Partial	Yes	Yes
Shelf listed in room order	No	Needs re-sorting	Yes	Free, by in-order walk

The gap is therefore a lightweight, zero-cost, fully offline tool that guarantees fair processing order and sub-linear retrieval at the same time. Most student implementations of this problem hold every record in one array or one linked list and accept O(n) search throughout; the contribution here is to match a structure to each access pattern and to demonstrate the difference by measurement.

1.6 Project Outcome

The delivered outcome is a working single-file C program of roughly 380 lines implementing six counter operations on two cooperating data structures. Intake and processing run in constant time per order; status lookup, delivery and removal run in expected logarithmic time; and the shelf is always displayed sorted by room number with no explicit sorting step. Measured on a tree of 3,200 bundles, a lookup needs about 14 key comparisons where a linear scan needs about 1,608, and 200,000 lookups complete 41 times faster. The program compiles without a single warning under -Wall -Wextra and reports no leak or undefined behaviour under AddressSanitizer.

Chapter 2
Proposed Methodology/Architecture

This chapter records the functional and non-functional requirements, presents the two-stage architecture and the reasoning behind each data-structure choice, shows the console interface, and sets out the plan by which the work was divided and completed.

2.1 Requirement Analysis & Design Specification

2.1.1 Overview

The requirements were collected by walking through the counter routine of a hall laundry service and writing down every action an attendant performs. They are listed in Table 2.1 and Table 2.2.

Table 2.1: Functional requirements

ID	Requirement	Realised by
FR1	Accept a new order with room number, guest name and piece count, and issue a unique token.	receiveOrder(), enqueue()
FR2	Hold pending orders in strict arrival order.	Linked-list queue
FR3	Process the oldest pending order and move it to storage.	processLaundry()
FR4	Report the status of any room, whether pending or finished.	checkStatus(), searchBST()
FR5	List all pending orders in arrival order and all stored bundles in room order.	viewQueue(), inorderPrint()
FR6	Hand a bundle to the guest, remove it from storage and print an itemised bill.	deliver(), deleteBST()
FR7	Compute the charge as piece count multiplied by the per-piece rate.	RATE constant

Table 2.2: Non-functional requirements

ID	Requirement
NFR1	The program must not terminate or corrupt its state on invalid input.
NFR2	Every allocated node must be freed; a long session must not grow in memory.
NFR3	Lookup cost must be sub-linear in the number of stored bundles.
NFR4	The program must build and run on both Windows and Linux consoles with no source change.
NFR5	Screens must be readable at a glance, with a consistent colour meaning for each type of message.

2.1.2 Proposed Methodology / System Design

The system is organised as a two-stage pipeline. Every order enters at the counter, waits in the pending queue, is dequeued exactly once for washing, and is then inserted into the storage tree, from which it is searched, listed and finally deleted at delivery. Figure 2.1 shows the flow of a bundle and the operations that connect the two structures.

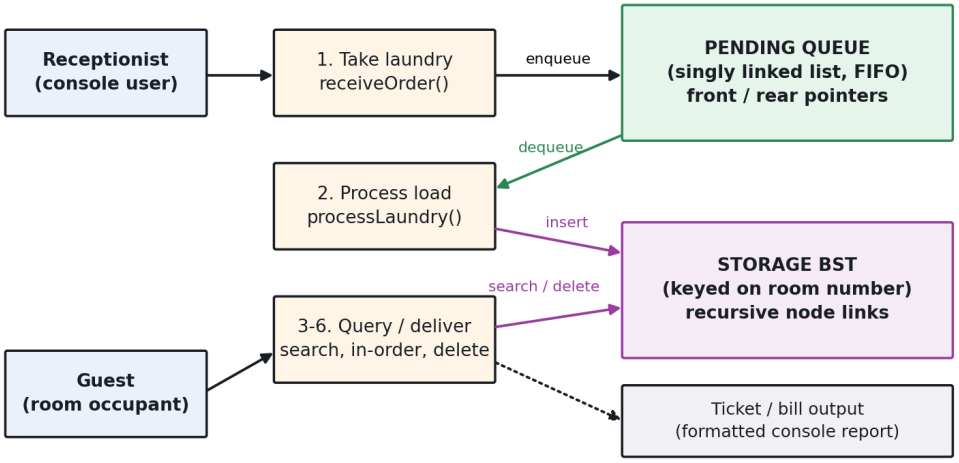


Figure 2.1: System architecture — a bundle moves from the FIFO queue into the storage BST

The design decision at the centre of the project is summarised in Table 2.3: each stage is given the structure whose cost profile matches the operation that dominates it.

Table 2.3: Justification of the data structure selected for each stage

Stage	Dominant operation	Structure	Reason
Intake	Append at the end	Linked-list queue	O(1) with a rear pointer; arrival order is preserved automatically, so FIFO fairness needs no extra logic.
Washing	Remove the oldest	Same queue	O(1) at the front; a dequeue is the only way an order can leave the queue, so no order can be skipped.
Storage	Search by room number	Binary search tree	Expected O(log n) search, insert and delete; the key is already a natural unique integer.
Shelf listing	Print in room order	Same tree	An in-order traversal yields sorted output in O(n) with no sorting step at all.

The record that travels through the pipeline is a single struct, so a bundle keeps its identity as it moves from one structure to the other. Both node types embed that record by value and add only their own links, as Table 2.4 shows.

Listing 2.1 The shared record and the two node types

```
typedef struct {                        typedef struct QueueNode {
    int  token;                             Laundry data;
    int  room_num;      /* BST key */       struct QueueNode *next;
    char name[50];                      } QueueNode;
    int  num_clothes;
    char status[24];                    typedef struct BSTNode {
} Laundry;                                  Laundry data;
                                            struct BSTNode *left, *right;
                                        } BSTNode;
```

Table 2.4: Memory footprint of one record (measured with sizeof on a 64-bit build)

Type	Size (bytes)	Composition
Laundry	88	3 ints, a 50-byte name, a 24-byte status field, plus padding
QueueNode	96	Laundry + one 8-byte next pointer
BSTNode	104	Laundry + two 8-byte child pointers

One thousand stored bundles therefore occupy about 104 KB, which is negligible for the target hardware. Listing 2.2 gives the deletion routine, the only non-trivial algorithm in the system, because removing a node with two children requires the value to be replaced by its in-order successor before the successor itself is removed.

Listing 2.2 Deletion from the storage tree, covering all three cases

```
BSTNode *deleteBST(BSTNode *root, int room) {
    if (!root) return NULL;
    if      (room < root->data.room_num) root->left  = deleteBST(root->left,  room);
    else if (room > root->data.room_num) root->right = deleteBST(root->right, room);
    else {                                      /* found the node to remove   */
        if (!root->left)  { BSTNode *r = root->right; free(root); return r; }
        if (!root->right) { BSTNode *l = root->left;  free(root); return l; }
        BSTNode *s = minNode(root->right);      /* in-order successor         */
        root->data  = s->data;                  /* copy it up ...             */
        root->right = deleteBST(root->right, s->data.room_num);  /* ... then  */
    }                                           /* delete it from the subtree */
    return root;
}
```

2.1.3 UI Design

The interface is a full-screen console layout built from a fixed 56-column frame drawn with Unicode box characters. Every screen repeats the same three regions: a dashboard banner reporting how many orders are pending and whether the shelf holds anything, the numbered menu, and then the panel produced by the chosen operation. Because the position of each region never changes, the operator learns the screen once.



Figure 2.2: The dashboard and main menu, redrawn after every operation

Colour carries meaning rather than decoration, which lets the attendant judge the outcome of an action before reading it. The scheme is given in Table 2.5.

Table 2.5: Meaning assigned to each colour in the interface

Colour	Used for	Example
Cyan	Frame, dashboard and input prompts	Banner, "Room number :"
Blue	The menu panel	Six numbered operations
Yellow	Work still pending	Pending queue, "still in queue"
Green	Completed and successful	Ticket, shelf listing, "ready for pickup"
Magenta	Money	The itemised bill at delivery
Red	Rejected input or missing record	"No record found for room 999."

Intake is the screen used most often, so it was designed to end in a self-contained ticket that can be read out to the guest, including the charge that will fall due on delivery.

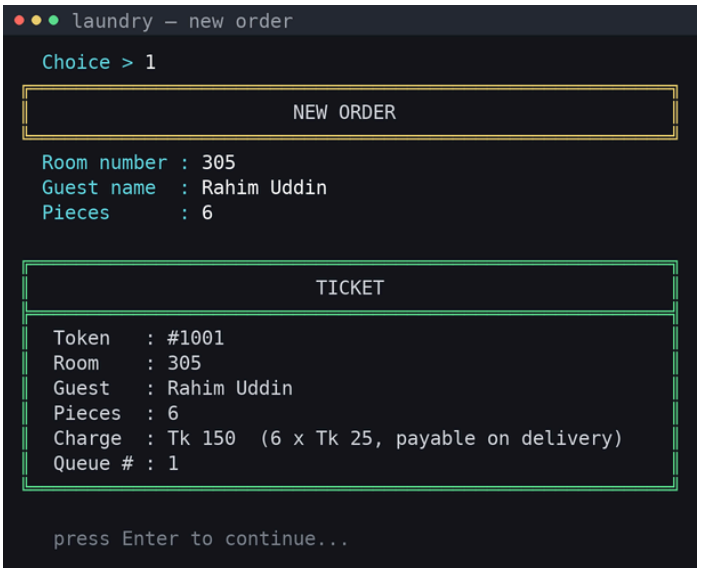


Figure 2.3: Order intake, ending in an automatically generated ticket

Two details of the implementation support NFR1 and NFR4. Numeric input is read through a wrapper that loops until a value is actually parsed and clears the rest of the line before returning, so a stray letter produces a polite message instead of an infinite loop; and an empty name defaults to "Guest" rather than leaving an uninitialised buffer. On Windows the program sets the console code page to UTF-8 and enables virtual-terminal processing at start-up, which is what allows the same box characters and colour codes to render on both platforms from one source file.

2.2 Overall Project Plan

The work was carried out by three members over five weeks. Table 2.6 records the plan that was followed; the division of responsibility is discussed further in section 4.2.

Table 2.6: Overall project plan

Week	Phase	Activity	Deliverable
1	Requirement analysis	Observed the counter routine, wrote FR1–FR7 and NFR1–NFR5	Requirement list
2	Design	Selected the queue and the tree, fixed the record layout and the menu	Architecture and UI sketch
3	Core implementation	Queue ADT, BST ADT, the dequeue-to-insert link	Working core
4	Interface and billing	Framed panels, colour scheme, ticket and bill, input validation	Complete program
5	Testing and analysis	Functional test cases, sanitizer run, comparison benchmark, report	Test log and report

Chapter 3
Implementation and Results

This chapter describes how the design was turned into code, analyses the cost of the two structures both asymptotically and by measurement, and presents the results of the functional test session.

3.1 Implementation

The system is one self-contained translation unit, so it can be handed to a lab machine and compiled in a single command. The environment is given in Table 3.1.

Table 3.1: Implementation environment

Item	Detail
Language	C (ISO C11), standard library only
Compiler	GCC with -Wall -Wextra -O2; also builds under MinGW-w64 on Windows
Source size	One file, approximately 380 lines
Dynamic memory	malloc for every node, free on dequeue, on BST delete and at shutdown
Platform layer	Conditional compilation on _WIN32 for sleep and console initialisation

Internally the program is layered: a presentation layer draws the frame, an input layer validates what the operator types, and two ADT layers implement the queue and the tree. The six menu operations sit on top and are the only place where the two ADTs meet, as Table 3.2 shows.

Table 3.2: Menu operations, the structure each one uses and its complexity

Menu	Function	Structure operation	Complexity	Effect
1	receiveOrder()	enqueue at rear	O(1)	New token issued, ticket printed
2	processLaundry()	dequeue + BST insert	O(1) + O(log n)	Oldest order washed and shelved
3	checkStatus()	BST search, then queue scan	O(log n) / O(m)	Reports stored, queued or unknown
4	viewQueue()	traverse the list	O(m)	Pending orders in arrival order
5	viewStorage()	in-order traversal	O(n)	Shelf sorted by room number
6	deliver()	BST search + delete	O(log n)	Bundle removed, bill printed

Here n is the number of stored bundles and m the number of pending orders. Listing 3.1 shows the queue operations, which keep both ends reachable in constant time; the rear pointer must be reset when the last element leaves, which is where a naive implementation typically leaves a dangling pointer.

Listing 3.1 Constant-time queue operations

```
static void enqueue(Laundry item) {          static int dequeue(Laundry *out) {
    QueueNode *n = malloc(sizeof *n);            if (!front) return 0;
    n->data = item; n->next = NULL;              QueueNode *t = front;
    if (!rear) { front = rear = n; return; }     *out  = t->data;
    rear->next = n;      /* link ... */          front = t->next;
    rear = n;            /* ... and advance */   if (!front) rear = NULL;
}                                                free(t);
                                                 return 1;
                                             }
```

The tree side is recursive throughout. Insertion returns the (possibly new) subtree root so the parent link is rebuilt on the way back up, removing the need for a parent pointer; searching follows the same comparison rule and stops when the key matches. An equal key overwrites the existing record, which is correct here because a room holds only one active bundle at a time.

Listing 3.2 Recursive insertion into the storage tree

```
static BSTNode *insertBST(BSTNode *root, Laundry item) {
    if (!root) {                          /* empty slot reached: attach   */
        BSTNode *n = malloc(sizeof *n);
        n->data = item; n->left = n->right = NULL;
```

```
        return n;
    }
    if      (item.room_num < root->data.room_num) root->left  = insertBST(root->left,  item);
    else if (item.room_num > root->data.room_num) root->right = insertBST(root->right, item);
    else root->data = item;                 /* same room: update in place   */
    return root;
}
```

Memory is released wherever a record leaves a structure: dequeue frees its node, each deletion case frees exactly the node unlinked, and on exit the queue is drained and the tree freed by a post-order walk. Post-order matters, because releasing the parent first would leave the child pointers unreachable.

3.2 Performance Analysis

The asymptotic cost of every operation is collected in Table 3.3. The queue is unconditionally constant time at both ends; the tree is logarithmic on average but degrades to linear if keys arrive already sorted, because each new key then becomes the right child of the previous one and the tree collapses into a list.

Table 3.3: Time and space complexity

Operation	Best	Average	Worst	Space
Queue enqueue / dequeue	Ω(1)	Θ(1)	O(1)	O(m)
Queue linear search	Ω(1)	Θ(m)	O(m)	O(1)
BST insert	Ω(1)	Θ(log n)	O(n)	O(n)
BST search / delete	Ω(1)	Θ(log n)	O(n)	O(1) aux
In-order traversal	Θ(n)	Θ(n)	Θ(n)	O(h) stack

To confirm the predicted behaviour, the queue and tree routines were compiled into an instrumented harness that counts key comparisons. For each size the room numbers were shuffled before insertion, 2,000 random lookups were issued and the average recorded; the experiment was repeated with keys inserted in sorted order to produce the degenerate tree. Results are in Table 3.4 and Figure 3.1.

Table 3.4: Measured comparisons per successful lookup

Bundles n	BST (shuffled)	BST (sorted input)	Linear scan of list	Tree height
25	4.91	13.09	13.08	8
100	6.78	50.19	49.40	11
400	9.85	203.55	201.73	17
1600	11.97	778.33	803.26	21
3200	13.99	1614.86	1608.01	25

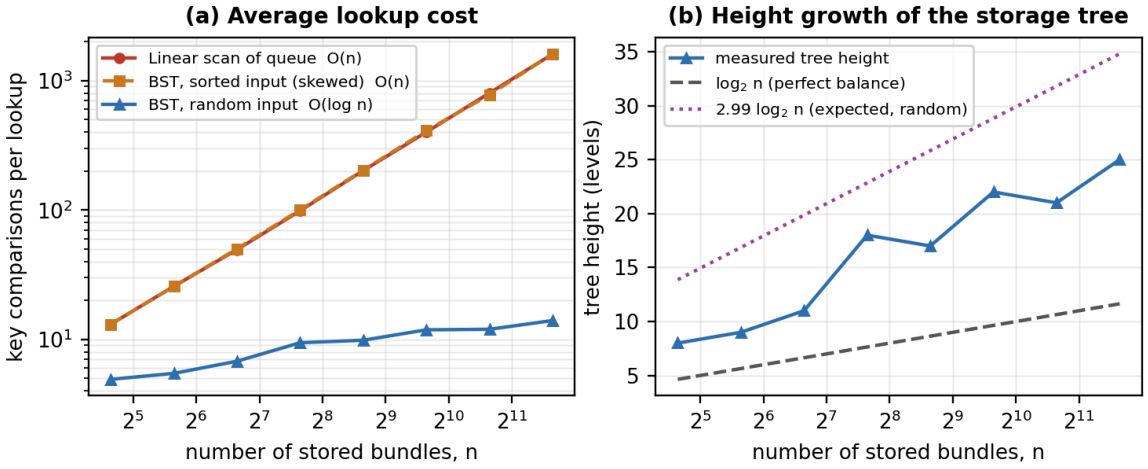


Figure 3.1: Measured lookup cost and tree height against the number of stored bundles

Three observations follow. First, the shuffled tree grows almost exactly as log n: multiplying the data by 128, from 25 bundles to 3,200, raises the average lookup from 4.91 comparisons only to 13.99, while the linear scan rises from 13.08 to 1,608.01 — a factor of 115 in favour of the tree. Second, the sorted-input curve lies on top of the linear-scan curve, the degenerate case predicted by theory and the price of an

unbalanced tree. Third, the measured height stays below the classical expected bound of about 2.99 log n for a randomly built tree [2], so the structure behaves as the analysis says on realistic input.

The comparison count translates directly into running time: over 200,000 lookups on a tree of 3,200 bundles, the tree took 0.016 s against 0.642 s for the linear scan, a speed-up of 41.3 times. At counter scale both are fast — a hall shelf holds tens of bundles, not thousands — so the practical justification for the tree is not raw speed but that its cost stops growing in any way the attendant would notice, and that the sorted shelf listing comes free with it.

Space cost is linear in the live records, at 96 bytes per pending order and 104 bytes per stored bundle, with nothing retained for delivered records. The recursive routines consume stack proportional to tree height, at most 25 frames in the largest experiment.

3.3 Results and Discussion

The system was exercised in one scripted session covering all six operations and both failure paths. Four orders were taken for rooms 305, 210, 415 and 118, three were processed, one bundle was delivered, and status was queried for a stored room, a pending room and a room that does not exist. Table 3.5 records each outcome.

Table 3.5: Functional test cases and results

ID	Test case	Expected result	Observed	Status
T1	Take four orders	Tokens 1001–1004 issued in order; queue depth reaches 4	As expected	Pass
T2	View pending queue	Listed front to rear as 305, 210, 415, 118	As expected	Pass
T3	Ticket charge	6 pieces at Tk 25 shown as Tk 150	Tk 150	Pass
T4	Process next load	Token 1001 (the oldest) leaves the queue first	Token 1001	Pass
T5	View shelf after three washes	Printed as 210, 305, 415 although washed 305, 210, 415	Sorted output	Pass
T6	Status of a stored room (305)	Reported ready with condition Washed & Ironed	As expected	Pass
T7	Status of a pending room (118)	Reported still in queue with its position	Position 1 of 1	Pass
T8	Deliver room 210	Bill of 4 × Tk 25 = Tk 100; node removed from the tree	Tk 100, removed	Pass
T9	Status of an unknown room (999)	Rejected with a clear message, no crash	As expected	Pass
T10	Non-numeric menu input	Re-prompt without entering an infinite loop	Re-prompted	Pass

Figure 3.2 shows the pending queue after the four orders were taken, confirming that arrival order is preserved exactly as entered.

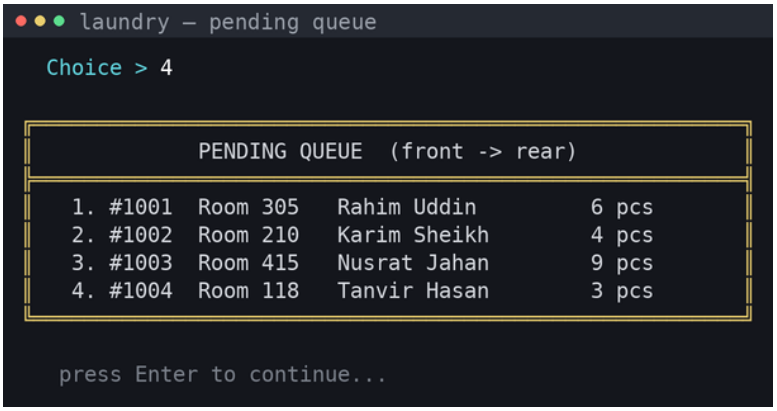


Figure 3.2: The pending queue, listed from front to rear (test case T2)

Processing removes the order at the front, runs the wash cycle and moves the record into the tree. Figure 3.3 shows selected frames for token 1001; the progress display keeps a batch operation from appearing frozen.

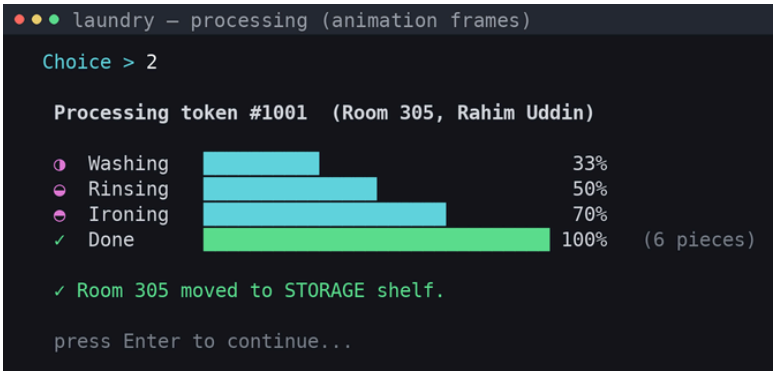


Figure 3.3: Processing the oldest order and transferring it to storage (test case T4)

The most instructive result is Figure 3.4. Rooms were washed in the order 305, 210, 415, yet the shelf prints as 210, 305, 415. No sort was ever called: the ordering is a property of the in-order traversal, obtained free from the structure chosen for searching.

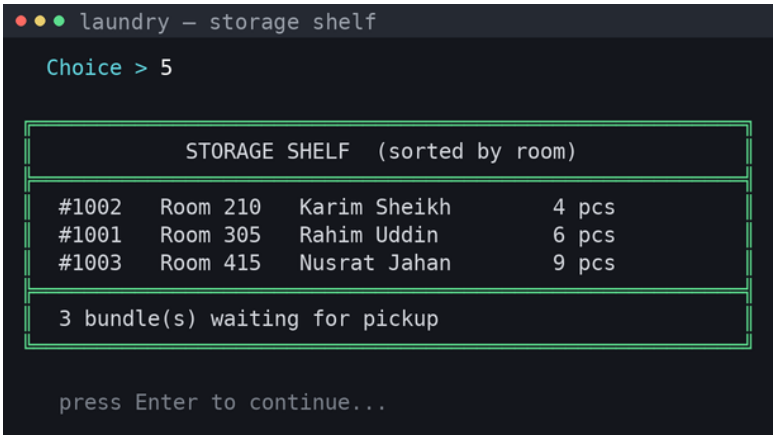


Figure 3.4: The storage shelf printed in room order by in-order traversal (test case T5)

Status lookup shows the two structures being queried in the right sequence: the tree is searched first, and only if the room is not on the shelf is the pending list scanned, which also lets the queue position be reported back as a waiting estimate.

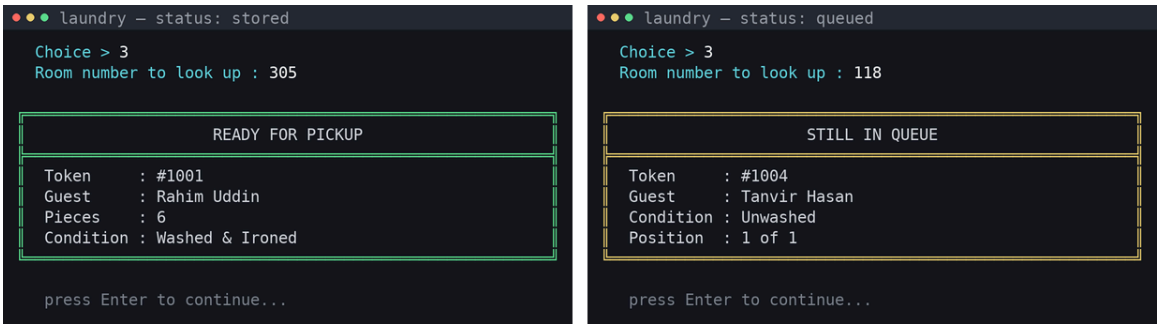


Figure 3.5: Status of a stored bundle (left, T6) and of one still waiting (right, T7)

Delivery closes the cycle: the record is found, removed by the deletion routine, and an itemised bill is produced from the piece count captured at intake, so the amount charged cannot drift from the amount agreed.

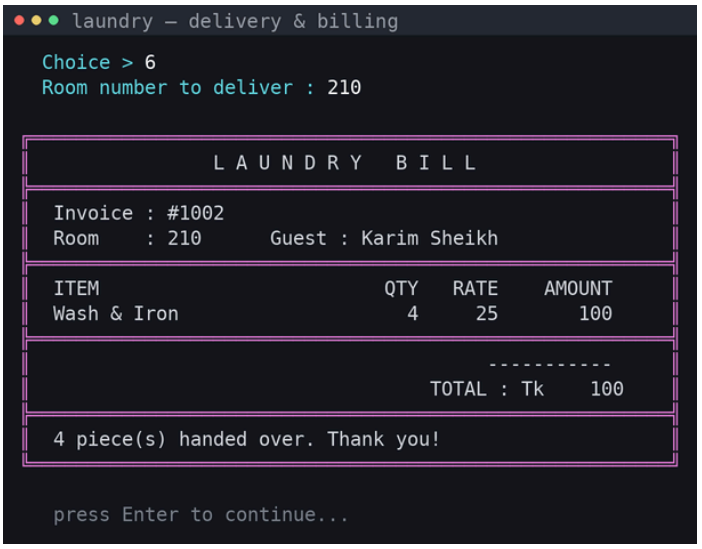


Figure 3.6: Itemised bill generated at delivery (test case T8)

Robustness was checked separately. Every menu path was executed with AddressSanitizer and UndefinedBehaviorSanitizer enabled; the session completed with no leak, no invalid access and no undefined behaviour reported, confirming NFR2, and the same build produced no diagnostics under -Wall -Wextra. Rooms in neither structure are rejected with a red message and the menu redrawn, satisfying NFR1.

Taken together the results confirm the central claim. Fairness is not enforced by special code — it follows from the queue, since a dequeue is the only exit from the pending stage. Ordered listing is not produced by sorting code — it follows from the tree. Choosing the right structure removed work rather than adding it.

Chapter 4
Engineering Standards and Mapping

This chapter examines the wider effects of the system, records how the team organised and costed the work, and maps the problem and its solution onto the programme outcomes, the complex-problem attributes and the complex engineering activities.

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

For the guest, the transaction becomes verifiable: a token issued at intake reappears on the bill with piece count and rate as separate columns, so a disagreement about the charge is settled by reading the ticket, and a status enquiry converts an open-ended wait into a position in the queue. For the attendant, the benefit is a reduction in remembering — the dashboard states how many orders are waiting, the next load is decided by the program rather than by recall, and the shelf is printed in room order.

4.1.2 Impact on Society & Environment

The social effect is modest but real: first-in-first-out processing is enforced by the structure of the program and cannot be quietly bypassed, which removes the suspicion of favouritism that manual queues invite in shared accommodation. The environmental effect should not be overstated — replacing a register and a book of slips removes a small continuing paper consumption, and the program runs on hardware that would otherwise be considered obsolete. No server, network equipment or continuous connection is needed, so the running energy cost is that of one desktop.

4.1.3 Ethical Aspects

The system stores a guest name against a room number, which is personal information. Records are held in memory only and discarded on exit, so nothing is written to disk that could later be copied; the same property means there is no audit trail, and any future move to file storage must bring access control with it. Only the data the service needs is captured — no phone number, no national identity number, no payment credential.

Billing transparency is an ethical requirement rather than a feature: the rate is one named constant applied uniformly, the bill shows quantity, rate and amount separately, and no code path can alter a charge after intake. Authorship is declared honestly — the algorithms are standard formulations from the references, the implementation and testing our own.

4.1.4 Sustainability Plan

Technically, the code is organised to survive being handed on: the queue and the tree are separated from the interface, so a future team can swap the unbalanced tree for an AVL or red-black tree, or replace the console layer with a graphical front end, without touching the other half, and with no external libraries the program will still compile years from now. Operationally there is no licence to renew and no vendor to depend on, and the one realistic maintenance need — file persistence — is confined to two functions, so the total cost of ownership after delivery is close to zero.

4.2 Project Management and Team Work

The team of three divided the work by module so the two data structures could be developed in parallel against an agreed record definition, and integrated in week three.

Table 4.1: Division of responsibility

Member	Primary responsibility	Contribution
MD Sajid Ali (253-15-147)	Queue ADT, presentation layer, integration	Linked-list queue, framed console layout and colour scheme, input validation, cross-platform setup
MD Junaid Al Habib Siam (253-15-769)	BST ADT, billing	Recursive insert, search, in-order traversal and the three deletion cases; ticket and bill
Manosh Sarkar (253-15-264)	Requirements, testing, documentation	Requirements, test design and execution, benchmark harness, report preparation

Progress was reviewed in a short meeting at the end of each week, where the current build was demonstrated and the next week's tasks agreed. Because all three members needed to understand both structures, each module was walked through line by line with the whole team once it was finished rather than being owned privately.

Cost analysis

Two budgets were prepared: the primary assumes in-house development on existing hardware, the alternate assumes a licensed commercial laundry point-of-sale product. Figures are indicative estimates in Taka for the first year.

Table 4.2: Primary budget — in-house development (year 1)

Item	Basis	Estimated cost
Development effort	3 members × 20 hours, coursework time	Tk 0 (unpaid)
Compiler and editor	GCC, MinGW, VS Code — all free	Tk 0
Hardware	Existing counter PC, no upgrade required	Tk 0
Electricity	One desktop, about 8 hours a day	Tk 2,400
Operator training	One session, in-house	Tk 0
Total		Tk 2,400

Table 4.3: Alternate budget — commercial laundry POS subscription (year 1)

Item	Basis	Estimated cost
Software subscription	Entry-tier laundry POS, monthly fee	Tk 18,000–36,000
Internet connectivity	Broadband at the counter	Tk 9,600
Hardware	Barcode printer and scanner, one-off	Tk 12,000
Vendor training and setup	One-off onboarding	Tk 5,000
Total		Tk 44,600–62,600

The rationale for the primary option is not only cost. A commercial product brings features the counter does not need — route delivery, multi-branch inventory, card payment — and one dependency it cannot absorb: a working internet connection at the moment a guest is at the desk. The alternate budget is still right at a different scale, since a chain with several outlets would outgrow this program quickly.

On the revenue side the counter charges Tk 25 per piece, the constant the program applies; at forty bundles a day of five pieces each it turns over roughly Tk 5,000 a day. The software does not create that revenue — its contribution is in what it prevents, namely unrecorded bundles, disputed piece counts and charges never collected — so even a small reduction in leakage repays the Tk 2,400 running cost many times over.

4.3 Complex Engineering Problem

4.3.1 Mapping of Program Outcome

Table 4.4: Justification of Program Outcomes

PO's	Justification
PO1	Engineering knowledge. The solution rests on the formal definitions of the FIFO queue and the binary search tree and on the asymptotic analysis of their operations. The queue was chosen from the definition of first-in-first-out access, the tree from the ordering property that makes a key comparison halve the remaining search space. The complexity claims in Table 3.3 are derived from that knowledge, not assumed.
PO2	Problem analysis. The workflow was decomposed until it became clear that it contains two distinct access patterns with conflicting requirements: strict arrival order for processing and arbitrary key access for retrieval. Recognising that no single structure satisfies both, and identifying dequeue-then-insert as the correct junction, is the analytical core of the project.

PO's	Justification
PO3	Design and development of solutions. A complete working system was built, with a defined record, layered ADTs, a validated interface, automatic billing and a shutdown path that releases all memory. Its effectiveness was evaluated against measured evidence — the comparison counts of Table 3.4, the 41.3× timing result and a clean sanitizer run — rather than asserted.

4.3.2 Complex Problem Solving

Table 4.5: Mapping with complex problem solving

EP1	EP2	EP3	EP4	EP5	EP6	EP7
Depth of Knowledge	Range of Conflicting Requirements	Depth of Analysis	Familiarity of Issues	Extent of Applicable Codes	Extent of Stakeholder Involvement	Interdependence
√	√	√				

EP1 — Depth of knowledge required

The solution cannot be reached from common sense or from experience of laundry counters; it requires the formal theory of abstract data types and their complexity. Knowing that the ordering property of a binary search tree yields sorted output from an in-order walk, and that the same tree degenerates under sorted insertion, is specialist knowledge applied deliberately. On the knowledge profile this is KP3, a theory-based formulation of engineering fundamentals, with KP4, engineering specialist knowledge, since section 3.2 draws on the theory of search trees.

EP2 — Range of conflicting requirements

The two central requirements pull in opposite directions. Fair processing demands strictly arrival-ordered access; retrieval demands that any record be reachable by key at any time. Strengthening either through one structure weakens the other, since an ordered structure cannot preserve arrival sequence and a queue cannot be searched efficiently. The conflict was resolved by staging rather than compromise, each requirement being given the structure that satisfies it with one controlled transfer between them; a further tension between readability and portability was settled by conditional compilation.

EP3 — Depth of analysis required

No obvious answer could be adopted without analysis. Alternatives were rejected on reasoned grounds: an array requires shifting on removal plus linear search or re-sorting; a sorted list gives ordered output but still linear search; a hash table gives fast lookup but destroys the ordered listing the shelf needs. Choosing between them required first-principles reasoning about the cost of each operation, then empirical validation including a deliberate test of the degenerate case.

4.3.3 Engineering Activities

Table 4.6: Mapping with complex engineering activities

EA1	EA2	EA3	EA4	EA5
Range of resources	Level of Interaction	Innovation	Consequences for society and environment	Familiarity
√	√	√		

EA1 — Range of resources

The work drew on several kinds of resource rather than code alone: the data structure literature, a toolchain with static warnings and runtime sanitizers, a purpose-written benchmark, and the time of three people over five weeks. Choosing the right instrument for each question — sanitizers for memory correctness, comparison counting for complexity — was part of the exercise.

EA2 — Level of interaction

The two subsystems were built by different members against a shared record definition and had to meet exactly at the transfer step, which required an agreed interface before either was written. Resolving the conflicts of section 4.3.2 needed judgement about which stage owns a record at any moment.

EA3 — Innovation

The innovation lies in the composition, not in the structures, which are standard. Staging a queue and a tree as one pipeline and exploiting the in-order traversal makes fairness and sorted output properties of the structure rather than coded features.

Chapter 5
Conclusion

This chapter summarises what was built and what was learned, states honestly what the system cannot do, and sets out the work that would follow.

5.1 Summary

This project implemented a laundry management system for a hall or small hotel counter as a two-stage pipeline in C. Incoming orders are held in a linked-list queue that guarantees first-in-first-out processing at constant cost; washed bundles move into a binary search tree keyed on room number, which gives expected logarithmic search, insertion and deletion and prints the shelf sorted through an in-order traversal. Around these two structures the program issues a unique token, validates every input, generates a ticket at intake and an itemised bill at delivery, and releases every node it allocates.

All ten functional test cases passed, the program compiled without warnings under -Wall -Wextra, and a full session under AddressSanitizer reported no leak or undefined behaviour. The measured comparison counts matched the predicted complexity closely: at 3,200 stored bundles a tree lookup needed 13.99 comparisons against 1,608.01 for a linear scan, and 200,000 lookups ran 41.3 times faster. The wider lesson is the one stated at the outset — matching each access pattern to the structure that suits it removed work from the implementation instead of adding it.

5.2 Limitation

The system has clear boundaries, and stating them accurately matters more than defending them.

- All data is held in memory. Closing the program, or a power cut, loses the day's records entirely; there is no persistence and therefore no audit trail.
- The storage tree is not self-balancing. If room numbers happen to be entered in ascending order the tree degenerates into a linked list and lookup falls back to O(n), as demonstrated deliberately in section 3.2.
- One room may hold only one active bundle. Inserting a second bundle for the same room overwrites the first, because the room number is the tree key.
- The program is single-user and has no authentication, so it assumes a trusted attendant at a single counter; it cannot be shared across two desks.
- Billing is a single flat rate per piece. There is no support for garment categories, express service, discounts or partial delivery of a bundle.
- The interface is text only and requires a console that supports UTF-8 and ANSI escape sequences; on a very old Windows console the frame characters may not render.

5.3 Future Work

The following extensions follow from the limitations above, ordered by practical value.

- Persistence. Serialise the queue and the tree to a file on exit and reload at start-up, so a session survives a restart and a daily record can be archived. This is the highest-value next step.
- A self-balancing tree. Replacing the plain BST with an AVL or red-black tree would guarantee O(log n) even under sorted insertion and would remove the only case where performance degrades; the ADT boundary already in place means the rest of the program would not change.
- Priority handling. An express service could be modelled with a priority queue so that urgent orders are processed ahead of ordinary ones without abandoning fairness within each class.
- Richer billing. Per-garment rates, express surcharges and partial delivery would require the record to hold a list of items rather than a single count.
- Notification. Sending an SMS when a bundle reaches the shelf would remove the need for a guest to enquire at all, and would make the status lookup a fallback rather than the main query path.
- Multi-counter operation. Moving the store behind a small database with user accounts would allow several desks to share one shelf, which is the point at which this design would need to be revisited rather than extended.

References

[1] J. Kleinberg and E. Tardos, Algorithm Design. Pearson Education India, 2006.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA: MIT Press, 2022.

[3] B. W. Kernighan and D. M. Ritchie, The C Programming Language, 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988.

[4] A. M. Tenenbaum, Y. Langsam and M. J. Augenstein, Data Structures Using C. Englewood Cliffs, NJ: Prentice Hall, 1990.

[5] D. E. Knuth, The Art of Computer Programming, Vol. 3: Sorting and Searching, 2nd ed. Reading, MA: Addison-Wesley, 1998.

[6] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Upper Saddle River, NJ: Addison-Wesley, 2011.

[7] ISO/IEC 9899:2018, Information technology — Programming languages — C. Geneva, Switzerland: International Organization for Standardization, 2018.

[8] M. A. Weiss, Data Structures and Algorithm Analysis in C, 2nd ed. Reading, MA: Addison-Wesley, 1997.